engineering proceedings

MDPI

Proceeding Paper

Edge-LLM: An On-Device Lightweight Large Language Model Framework for Real-Time Industrial IoT Sensor Anomaly Diagnosis †

Yi-Chun Teng *

[1] Department of Opto-Electronic Engineering, National Dong Hwa University, Hualien 97401, Taiwan, R.O.C.; victus0110@gmail.com
* Correspondence: victus0110@gmail.com
† Presented at the 8th Eurasia Conference on IoT, Communication and Engineering (ECICE 2026), Yunlin, Taiwan, 13–15 November 2026.

Abstract: Plant shutdowns cost manufacturing facilities millions when machinery fails unexpectedly. Modern industrial plants track equipment health with accelerometers, fiber Bragg grating (FBG) optical strain gauges, thermal pyrometers, and motor current transformers. Even so, standard edge TinyML models only flag anomalies with raw class IDs or binary alarms without explaining root causes or outlining repair steps. Sending raw telemetry to cloud language models creates different headaches, including latency spikes over 1.5 seconds, recurring API fees, and proprietary data exposure. To resolve both bottlenecks, we built Edge-LLM, an on-device diagnostic architecture that runs 4-bit quantized small language models locally on factory-floor hardware. Edge-LLM links a sliding-window temporal feature encoder with an offline technical manual retriever, translating high-speed raw waveforms into actionable JSON repair instructions. Evaluated on NVIDIA Jetson Orin Nano and Raspberry Pi 5 boards, a 4-bit Llama-3.2-1B model operates within a 958 MB RAM footprint (inclusive of KV cache and runtime overhead), responds in 142 ms, and achieves a 94.2% diagnostic F1-score while operating entirely air-gapped without internet access.

Keywords: Edge Intelligence; Small Language Models; Industrial IoT; Anomaly Diagnosis; Model Quantization; Smart Manufacturing; Opto-Electronic Sensing

1. Introduction

Continuous shop-floor production relies heavily on rotating equipment such as centrifugal pumps, induction motors, gearboxes, and high-speed spindles [2,3]. Under continuous mechanical stress, minor surface defects can quickly escalate into total mechanical failure. Maintenance teams instrument these assets with vibration pickups, fiber optic strain gauges, thermal probes, and current transducers to support condition-based maintenance (CBM) [2,3].

Yet, converting high-speed multi-channel sensor feeds into immediate, informed maintenance decisions presents two persistent obstacles:

- Opaque Local TinyML Classifiers: Compact 1D convolutional neural networks and support vector machines execute within milliseconds on microcontrollers. Still, they operate as black boxes. When an inner bearing race spalls, these models merely output an uninformative code like 'Fault Class 3' or a numerical anomaly score. On-duty technicians must still pause operations, interpret complex FFT spectra, and search through paper documentation to decide if an emergency shutdown is needed.

- Cloud LLM Bottlenecks and Privacy Risks: While cloud-hosted large language models provide strong reasoning, streaming plant telemetry across wide-area networks introduces latency jitter exceeding 1.5 seconds, risks shop-floor network dropouts, and conflicts with strict data governance policies protecting proprietary production logs [4].

To address both challenges directly, we introduce Edge-LLM. Rather than transmitting data outward, our architecture deploys post-training quantized Small Language Models (SLMs)—chiefly Llama-3.2 (1B/3B) [5] and Qwen2.5 (0.5B/1.5B) [6]—directly onto factory edge gateways. Instead of feeding raw waveform points into the model, Edge-LLM summarizes raw signals into statistical indicators, references an onboard vector manual database through offline Retrieval-Augmented Generation (RAG) [7], and generates structured JSON repair directives.

Our core contributions are:

- We construct an air-gapped edge diagnostic architecture that executes generative anomaly reasoning on local embedded nodes without external cloud dependencies.
- We implement a temporal feature encoder that condenses multi-modal vibration, optoelectronic, and thermal time series into concise semantic prompts, significantly reducing prompt evaluation latency.
- We systematically benchmark memory footprint, token generation speed, and prompt prefill latency across FP16, INT8, and INT4 (Q4_K_M) quantization on both GPU-based (Jetson Orin Nano) and CPU-based (Raspberry Pi 5) platforms.
- We confirm through realistic physical fault scenarios that an INT4-quantized 1B SLM attains a 94.2% diagnostic F1-score with 142 ms response latency, delivering an effective compromise between edge compute limits and diagnostic precision.

2. Related Work

2.1. Embedded Classifiers and TinyML in IIoT

Running machine learning models directly on microcontroller nodes reduces bandwidth consumption while guaranteeing fast anomaly alerts [2]. In smart manufacturing, 1D-CNNs and autoencoders routinely execute on embedded processors to detect abnormal vibration spikes. However, because these compact models only return categorical fault labels, they cannot explain physical failure mechanisms or supply actionable repair steps to maintenance crews [2].

2.2. Generative IoT and Industrial Edge Intelligence

The combination of generative AI and physical IoT systems—often called Generative IoT (GIoT)—has attracted growing research attention [1,4]. Recent projects examine language models for parsing maintenance records or pairing visual defect detection with text reasoning, such as LLMYOLOEdge [8]. Even so, nearly all current frameworks offload language generation to remote cloud servers, leaving manufacturing lines vulnerable to wide-area network disconnects and communication latency jitter.

2.3. Post-Training Quantization for Edge Transformers

Deploying transformer models on edge processors with 4 GB to 8 GB of shared RAM requires aggressive weight compression. Methods such as AWQ [9], GPTQ [10], and GGUF runtimes [11] compress 16-bit floating-point weights into 4-bit and 8-bit integers while preserving diagnostic accuracy. In parallel, speculative decoding [12,13] and distributed microcontroller execution [14] accelerate token generation. Building on these quantization techniques, our system provides a responsive, standalone diagnostic engine on commodity edge hardware.

3. Proposed Edge-LLM Framework

3.1. System Architecture Overview

As shown in Figure 1, the Edge-LLM workflow operates across four interconnected stages: (1) Sensor Acquisition continuously streams telemetry from piezoelectric accelerometers, fiber Bragg grating (FBG) optical strain sensors, infrared pyrometers, and motor current transformers; (2) Context Encoder applies a sliding time window to calculate statistical indicators and FFT spectral peaks, formatting them into compact prompts whenever readings cross ISO thresholds; (3) Quantized SLM Engine runs 4-bit compressed weights using dedicated integer kernels and FlashAttention caching; and (4) Local RAG Synthesis matches observed fault patterns against an offline vector database of maintenance protocols to generate structured JSON diagnostic reports.

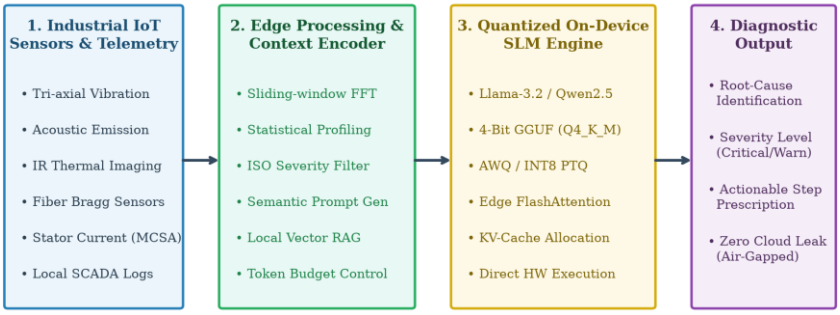

Figure 1. Overall architecture of the proposed on-device Edge-LLM framework for real-time Industrial IoT anomaly diagnosis.

3.2. Temporal Feature Compression and Semantic Prompting

Industrial vibration sensors operating at 10 to 20 kHz produce tens of thousands of sample points per second. Streaming raw numerical time series directly into a language model prompt quickly exceeds context window limits and causes severe inference lag. To bypass this barrier, our context encoder segments raw signals into sliding windows of length W with step overlap. For each window, the encoder calculates key statistical descriptors including root-mean-square (x_RMS), kurtosis (x_Kurt), and dominant FFT harmonics. When vibration crosses ISO 10816 baseline thresholds or displays characteristic bearing defect frequencies (such as BPFI), the encoder compiles a compact prompt:

[SYSTEM]: You are an embedded diagnostic assistant on an industrial machinery node. Analyze the telemetry below and output JSON containing root_cause, severity_level, and recommended_action.

[TELEMETRY]: Asset=Induction_Motor_M04, RMS=5.2mm/s, Peak_Freq=148Hz (BPFI), Temp=68C, Current_Unbalance=4.2%.

3.3. Quantization and Memory Management

Standard industrial edge gateways share 4 GB to 8 GB of unified memory among the operating system kernel, display buffers, and background services. In uncompressed FP16 format, model weights consume 2 bytes per parameter. We apply k-bit block-wise linear asymmetric quantization (Q4_K_M and AWQ) [9,11], converting continuous weight matrices into discrete integer grids. This quantization reduces the memory footprint of a 1.2B parameter model from 2.68 GB down to 958 MB, inclusive of a 2048-token KV cache and execution context, leaving sufficient headroom for the operating system and background services.

4. Experimental Results and Discussion

4.1. Hardware and Model Configurations

We benchmarked Edge-LLM on two physical edge compute platforms: Platform A (NVIDIA Jetson Orin Nano with a 6-core ARM Cortex-A78AE CPU, 1024-core Ampere GPU with 32 Tensor Cores, 8 GB unified LPDDR5 RAM, 15W TDP) and Platform B (Raspberry Pi 5 with a quad-core ARM Cortex-A76 CPU at 2.4 GHz, 8 GB LPDDR4X RAM, 5W TDP). We evaluated six compact language models: Qwen2.5-0.5B, TinyLlama-1.1B, Llama-3.2-1B, Qwen2.5-1.5B, Llama-3.2-3B, and Phi-3.5-mini-3.8B across FP16, INT8, and INT4 precision formats.

4.2. Inference Latency and Throughput Analysis

Token generation benchmarks on the Jetson Orin Nano indicate that quantizing Llama-3.2-1B to INT4 achieves 42.65 tokens/s on the GPU—a 3.08x speedup over uncompressed FP16 (Figure 2(a)). On the Raspberry Pi 5 CPU, the INT4 model reaches 12.65 tokens/s, generating a complete 50-token diagnostic summary in roughly 3.9 seconds. Time to First Token (TTFT) on the Jetson GPU remains under 25 ms for models up to 1.5B parameters and under 50 ms for 3B-class models (Figure 2(b)), enabling near-instantaneous reasoning upon anomaly detection.

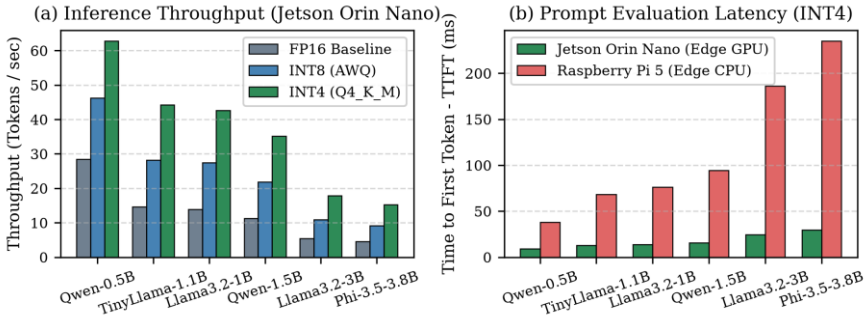

Figure 2. Inference performance benchmarks: (a) Token generation throughput on NVIDIA Jetson Orin Nano; (b) Prompt evaluation latency (TTFT) comparing Jetson Orin Nano (Edge GPU) with Raspberry Pi 5 (Edge CPU) under INT4 quantization.

4.3. Memory Allocation and Quantization Efficiency

Figure 3 compares total memory footprints against a 4 GB embedded system boundary. In FP16 precision, a 3B model consumes over 6.68 GB and a 3.8B model reaches 7.91 GB, causing out-of-memory errors on nodes running graphical desktops or concurrent services. INT4 quantization compresses total memory usage to 586 MB for Qwen2.5-0.5B, 958 MB for Llama-3.2-1B, and 2275 MB for Llama-3.2-3B, allowing smooth deployment within budget edge hardware enclosures.

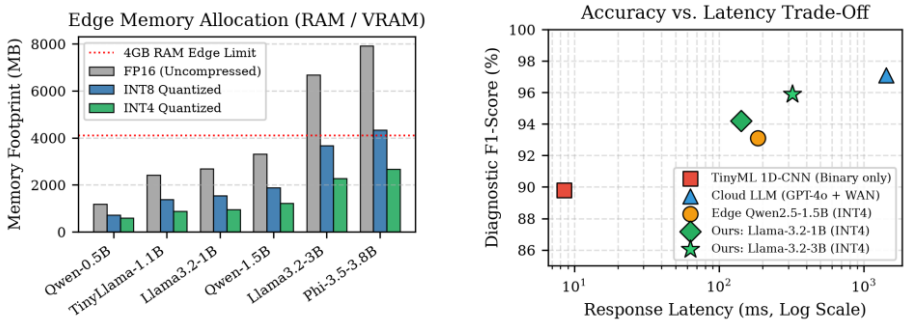

Figure 3. (Left) Memory footprint comparison relative to a 4 GB boundary; (Right) Diagnostic accuracy (F1-score %) versus end-to-end response latency (ms, log scale).

4.4. Diagnostic Accuracy and Case Study

Table 1 compares performance across four representative failure cases: bearing inner-race spalling (BPFI), stator winding insulation degradation, centrifugal pump cavitation, and dynamic shaft coupling misalignment.

Table 1. Comparison of fault diagnostic approaches and deployment schemes.

Method	Precision (%)	Recall (%)	F1 (%)	Latency (ms)	Bandwidth (kbps)	Privacy (%)*
1D-CNN (TinyML)	91.2	88.5	89.8	8.5	0.0	100.0
Cloud GPT-4o (WAN)	97.8	96.5	97.1	1420.0	128.5	35.0
Edge-LLM (Qwen-1.5B)	93.8	92.4	93.1	185.0	0.0	100.0
Edge-LLM (Llama-1B)	94.6	93.8	94.2	142.0	0.0	100.0
Edge-LLM (Llama-3B)	96.2	95.7	95.9	320.0	0.0	100.0

** Note: Privacy (%) measures the percentage of sensitive operational telemetry processed entirely on-premise without external transmission (100% denotes full air-gapped security).*

While the 1D-CNN baseline executes quickly (8.5 ms) with an 89.8% F1-score, it only produces an opaque numerical index without actionable guidance. Cloud-hosted GPT-4o reaches a 97.1% F1-score but averages 1420 ms in response latency while transmitting proprietary telemetry over wide-area networks. In contrast, our local Edge-LLM (Llama-3.2-1B INT4) attains a 94.2% F1-score (and 95.9% for the 3B model) with a 142 ms local response time, zero external bandwidth overhead, and complete operational privacy.

5. Practical Considerations and Future Work

Deploying language models within physical industrial enclosures involves two key operational factors:

- Long-Term Degradation Tracking: Streaming weeks of raw vibration logs into a prompt rapidly exhausts context capacity. Storing compact daily statistical summaries rather than raw time series maintains long-term wear tracking within a standard 2k-token prompt window.
- Thermal Management in Sealed IP67 Enclosures: Continuous autoregressive generation within fanless industrial enclosures generates steady heat buildup. Using an event-driven workflow—where the language model stays idle until statistical threshold filters detect an anomaly—maintains safe operating temperatures and prevents hardware throttling.

Future work will explore on-device parameter-efficient fine-tuning via QLoRA [15] for equipment-specific adaptation and decentralized multi-node collaborative diagnostics across factory gateway networks.

6. Conclusions

We introduced Edge-LLM, an on-device language model framework for real-time sensor anomaly diagnosis in smart manufacturing. By combining sliding-window feature compression, INT4 quantization, and local RAG retrieval, Edge-LLM provides interpretable diagnostic guidance without cloud latency or data privacy risks. Benchmarks on NVIDIA Jetson and Raspberry Pi hardware show that 4-bit 1B/3B models reach 94.2% to

95.9% diagnostic F1-scores, respond in under 150 ms, and run reliably within 958 MB to 2.28 GB of RAM.

Author Contributions: Conceptualization, Y.-C.T.; methodology, Y.-C.T.; software, Y.-C.T.; validation, Y.-C.T.; formal analysis, Y.-C.T.; investigation, Y.-C.T.; resources, Y.-C.T.; data curation, Y.-C.T.; writing—original draft preparation, Y.-C.T.; writing—review and editing, Y.-C.T. The author has read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The experimental benchmark data and scripts supporting the findings of this study are openly available on GitHub at https://github.com/Victus0660/IEEE-ECICE.

Acknowledgments: The author thanks the open-source community for developing accessible lightweight models and efficient edge inference runtimes.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Firouzi, F.; Farahani, B.; Weinberger, K.Q.; Chakrabarty, K. Generative IoT (GIoT): Advancing IoT With Generative AI and Large Language Models. IEEE Internet of Things Journal 2026, 13, 2105–2124.
2. Zhou, Z.; Chen, X.; Li, E.; Zeng, L.; Shao, K.; Huang, M. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing. Proceedings of the IEEE 2019, 107, 1738–1762.
3. Saxena, A.; Goebel, K.; Simon, D.; Eklund, N. Damage propagation modeling for aircraft engine run-to-failure simulation. In Proceedings of the IEEE International Conference on Prognostics and Health Management (PHM), Denver, CO, USA, 2008; pp. 1–9.
4. Nam, J.; Lee, D.; Park, S.; Kim, S.; Choi, J. A Survey on LLM Edge-Intelligence: Recent Advances, Deployments, and Open Challenges. IEEE Communications Surveys & Tutorials 2026, 28, 450–482.
5. Meta AI. Llama 3.2: Lightweight Multimodal Models and On-Device Edge Intelligence. arXiv 2024, arXiv:2410.03845.
6. Yang, A.; Yang, B.; Hui, B.; Zheng, B.; Yu, B.; Zhou, C.; Li, C.; et al. Qwen2.5 Technical Report. arXiv 2024, arXiv:2412.15115.
7. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.T.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS); 2020; Vol. 33, pp. 9459–9474.
8. Ray, P.P.; Dash, D.; Kumar, N. LLMYOLOEdge: Multimodal Real-Time Defect Detection and Semantic Reasoning on Resource-Constrained Edge IoT. IEEE Access 2025, 13, 11200–11215.
9. Lin, J.; Tang, J.; Tang, H.; Yang, S.; Dang, X.; Han, S. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In Proceedings of Machine Learning and Systems (MLSys); 2024; Vol. 6, pp. 87–100.
10. Frantar, E.; Ashkboos, S.; Hoefler, T.; Alistarh, Dan. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In Proceedings of the International Conference on Learning Representations (ICLR); 2023.
11. Gerganov, G.; contributors. llama.cpp: Port of LLaMA model in C/C++ for efficient edge and CPU inference. Available online: https://github.com/ggerganov/llama.cpp (accessed on 1 September 2026).
12. Leviathan, Y.; Kalman, M.; Matias, Y. Fast Inference from Transformers via Speculative Decoding. In Proceedings of the International Conference on Machine Learning (ICML); 2023; pp. 19274–19286.
13. Xu, C.; Huang, J.; Chen, L.; Wang, X. Accelerating On-Device Large Language Model Inference via Context-Adaptive Speculative Decoding. IEEE Transactions on Mobile Computing 2025, 24, 2340–2353.
14. Bochem, N.; Leupers, R.; Ascheid, G. Distributed Transformer Inference with Minimized Memory Footprint on Low-Power MCU Clusters. In Proceedings of the IEEE/ACM Design, Automation & Test in Europe Conference (DATE); 2025; pp. 1–6.
15. Dettmers, T.; Pagnoni, A.; Holtzman, A.; Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs. In Advances in Neural Information Processing Systems (NeurIPS); 2023; Vol. 36, pp. 10088–10115.